Agora Architecture Design Document

> Multi-agent collaboration platform -- games are the wedge, general-purpose is the goal.

Table of Contents

1. System Overview
2. Package Structure
3. Core Abstractions
4. Mode System
5. Frontend Architecture
6. LLM Integration
7. Data Model
8. Real-time Communication
9. Memory System Deep Dive
10. Security & Performance Considerations

1. System Overview

Three-Layer Architecture

```
+==========================================================================
|                              MODE LAYER
|   [Roundtable]  [Werewolf]  [Script Kill]  [TRPG]  [Custom...]
|
|   Each mode provides:
|     - Role templates        - Channel rules
|     - Flow configuration    - Decision schemas (Zod)
|     - UI extensions         - Lifecycle hooks
+==========================================================================
        |               |               |               |
        v               v               v               v
+==========================================================================
|                           PLATFORM CORE
|
|   +-------+  +------+  +---------+  +----------------+  +---------+
|   | Agent |  | Room |  | Channel |  | FlowController |  | Memory  |
|   +-------+  +------+  +---------+  +----------------+  +---------+
|
|   +----------+  +------------------+  +---------+
|   | EventBus |  | StructuredOutput |  | Message |
|   +----------+  +------------------+  +---------+
+==========================================================================
        |               |               |               |
        v               v               v               v
+==========================================================================
|                          INFRASTRUCTURE
|
|   [Vercel AI SDK]  [Socket.io]  [Postgres/pgvector]  [Next.js 15]
|   [Turborepo]      [Supabase]   [Zod]
+==========================================================================
```

Data Flow

```
User creates Room
       |
       v
Room configured (mode, agent count, topic/scenario)
       |
       v
Agents join Room (each with persona, model binding, memory)
       |
       v
Mode starts --> FlowController initialized with mode's flow config
       |
       v
FlowController selects next speaker(s)
       |
       v
Agent.reply() called --> LLM generates response (structured via Zod)
       |
       v
Message created --> routed through Channel system
       |
       v
Channel applies visibility masks --> eligible agents receive via observe
       |
       v
EventBus emits events --> Socket.io pushes to subscribed frontend client
       |
       v
UI updates (chat feed, phase indicator, vote panel, etc.)
       |
       v
FlowController checks transition conditions --> phase/state changes
       |
       v
Loop continues until mode signals completion
```

2. Package Structure

```
agora/
├── apps/
│   └── web/                    # Next.js 15 frontend (App Router)
│       ├── app/
│       │   ├── (lobby)/        # Room list, create room
│       │   ├── room/[id]/      # Room detail, game view
│       │   └── api/            # Next.js API routes (REST + Socket.io)
│       ├── components/         # Shared UI components
│       ├── hooks/              # React hooks (useRoom, useSocket, useAg
│       └── lib/                # Frontend utilities
│
├── packages/
│   ├── core/                   # Platform core (mode-agnostic)
│   │   ├── agent/              # Agent interface + base implementation
│   │   ├── room/               # Room lifecycle management
│   │   ├── channel/            # Channel system (visibility, scoping)
│   │   ├── flow/               # FlowController interface + implementat
│   │   ├── memory/             # SessionMemory + AgentLongTermMemory
│   │   ├── message/            # Message schema and routing
│   │   └── events/             # EventBus for platform events
│   │
│   ├── modes/                  # Mode plugins
│   │   ├── roundtable/         # Roundtable debate mode
│   │   ├── werewolf/           # Werewolf (狼人杀) mode
│   │   ├── script-kill/        # Script Kill (剧本杀) mode
│   │   └── trpg/               # TRPG (跑团) mode
│   │
│   ├── llm/                    # Vercel AI SDK wrapper
│   │   ├── provider.ts         # Multi-model provider registry
│   │   ├── generate.ts         # Text + structured generation
│   │   ├── stream.ts           # Streaming utilities
│   │   └── schemas/            # Shared Zod schemas for decisions
│   │
│   └── shared/                 # Types, constants, utilities
│       ├── types/              # Shared TypeScript types
│       ├── constants/          # Platform-wide constants
│       └── utils/              # Pure utility functions
│
├── turbo.json                  # Turborepo pipeline config
├── package.json                # Root workspace config
└── tsconfig.base.json          # Shared TypeScript config
```

Package Responsibilities

Package	Layer	Responsibility
`apps/web`	Infrastructure + UI	Next.js frontend, API routes, Socket.io server, page rendering
`packages/core`	Platform Core	All mode-agnostic abstractions: Agent, Room, Channel, FlowController, Memory, EventBus, Message
`packages/modes`	Mode Layer	Pluggable interaction modes, each self-contained with flow config, role templates, channel rules, schemas, UI extensions
`packages/llm`	Infrastructure	Vercel AI SDK wrapper, multi-model routing, structured output generation, streaming
`packages/shared`	Cross-cutting	TypeScript types shared across all packages, constants, pure utility functions

Dependency Graph

```
apps/web --> packages/core, packages/modes, packages/llm, packages/share
packages/core --> packages/llm, packages/shared
packages/modes --> packages/core, packages/shared
packages/llm --> packages/shared
packages/shared --> (no internal deps)
```

3. Core Abstractions

3.1 Message

The foundational data unit. Defined first because every other abstraction references it.

```typescript
// packages/shared/types/message.ts

type MessageRole = 'agent' | 'system' | 'human' | 'narrator';

interface MessageVisibility {
  /** If set, only these agent IDs can see this message. Empty = nobody.
  readonly allowList?: readonly string[];
  /** If set, these agent IDs cannot see this message. */
  readonly denyList?: readonly string[];
  /** If true, message is visible to all channel subscribers (default).
  readonly broadcast: boolean;
}

interface MessageMetadata {
  /** Structured decision data (vote result, action choice, etc.) */
  readonly structured?: Record<string, unknown>;
  /** Phase/state when message was sent */
  readonly phase?: string;
  /** If this message is a reply to another */
  readonly replyTo?: string;
  /** Token count for cost tracking */
  readonly tokenCount?: number;
  /** Custom mode-specific metadata */
  readonly [key: string]: unknown;
}

interface Message {
  readonly id: string;
  readonly roomId: string;
  readonly senderId: string;
  readonly senderName: string;
  readonly role: MessageRole;
  readonly content: string;
  readonly channelId: string;
  readonly visibility: MessageVisibility;
  readonly metadata: MessageMetadata;
  readonly createdAt: Date;
}
```

Design decisions: - Immutable (`readonly` throughout). Messages are facts -- once created, never mutated. - `visibility` is per-message, not per-channel. A channel has default visibility, but individual messages can override it (e.g., a whisper within a public channel). - `metadata.structured` holds Zod-validated decision data (votes, actions) alongside the natural language `content` . This lets the UI render rich components while agents process structured data. - `channelId` determines routing. The Channel system uses this to decide who receives the message.

3.2 Agent

Borrowed from AgentScope's `AgentBase` two-method pattern: `reply()` produces output, `observe()` consumes input.

```
// packages/core/agent/types.ts

interface AgentConfig {
  readonly id: string;
  readonly name: string;
  readonly persona: string;
  readonly modelId: string;           // e.g., 'claude-sonnet-4-20250514
  readonly systemPrompt?: string;     // Auto-generated from persona + m
  readonly temperature?: number;
  readonly maxTokensPerTurn?: number;
  readonly isHuman?: boolean;         // Human-controlled agent (no LLM
}

interface AgentState {
  readonly id: string;
  readonly config: AgentConfig;
  readonly roomId: string | null;
  readonly role: string | null;       // Mode-specific role (e.g., 'were
  readonly status: 'idle' | 'thinking' | 'speaking' | 'observing' | 'eli
  readonly channelIds: readonly string[];  // Channels this agent is sub
  readonly memory: SessionMemory;
  readonly longTermMemory: AgentLongTermMemory | null;
  readonly metadata: Record<string, unknown>;  // Mode-specific state
}

interface Agent {
  readonly state: AgentState;

  /**
   * Produce a response given the current context.
   * The FlowController calls this when it's the agent's turn.
   *
   * @param context - Current room state, recent messages, phase info
   * @param schema  - Optional Zod schema to constrain output (for decis
   * @returns The agent's message (content + optional structured data)
   */
  reply(context: ReplyContext, schema?: ZodSchema): Promise<Message>;

  /**
   * Receive a message from the channel system.
   * Called by the Channel when a message is broadcast to this agent.
   * Updates the agent's session memory.
```

```
   *
   * @param message - The incoming message
   */
  observe(message: Message): Promise<void>;

  /**
   * Update agent state (role assignment, channel subscription, etc.)
   * Returns a new AgentState (immutable update).
   */
  updateState(patch: Partial<AgentState>): AgentState;
}

interface ReplyContext {
  readonly roomId: string;
  readonly phase: string;
  readonly recentMessages: readonly Message[];
  readonly visibleAgents: readonly AgentConfig[];
  readonly modeContext: Record<string, unknown>;   // Mode-specific cont
  readonly instruction?: string;                    // Per-turn instruct
}
```

Design decisions: - `reply()` + `observe()` is the entire agent contract. A mode never calls agents any other way. This makes agents pluggable and testable. - `reply()` accepts an optional Zod schema. When provided, the LLM layer uses structured output (tool calling or JSON mode) to enforce the schema. This replaces free-text parsing for votes, decisions, and actions. - `observe()` is fire-and-forget from the channel's perspective. The agent internally decides whether to store, summarize, or ignore the message. - Human agents implement the same interface. `reply()` for a human agent waits for Socket.io input instead of calling an LLM. The rest of the system is unaware of the difference. - `AgentState` is immutable. `updateState()` returns a new state object.

3.3 Room

The top-level container for a collaboration session.

```
// packages/core/room/types.ts

type RoomStatus = 'waiting' | 'running' | 'paused' | 'completed';

interface RoomConfig {
  readonly id: string;
  readonly name: string;
  readonly modeId: string;              // Which mode plugin to use
  readonly modeConfig: Record<string, unknown>;  // Mode-specific settir
  readonly maxAgents: number;
  readonly allowHumans: boolean;
  readonly createdBy: string;           // User ID
  readonly createdAt: Date;
}

interface RoomState {
  readonly config: RoomConfig;
  readonly status: RoomStatus;
  readonly agents: readonly AgentState[];
  readonly channels: readonly ChannelState[];
  readonly currentPhase: string;
  readonly phaseData: Record<string, unknown>;  // Mode-specific phase s
  readonly startedAt: Date | null;
  readonly completedAt: Date | null;
}

interface Room {
  readonly state: RoomState;

  /** Add an agent to the room. Fails if room is full or running. */
  addAgent(config: AgentConfig): RoomState;

  /** Remove an agent from the room. */
  removeAgent(agentId: string): RoomState;

  /** Start the mode. Initializes FlowController, creates channels, assi
  start(): Promise<RoomState>;

  /** Pause execution (agents stop taking turns). */
  pause(): RoomState;

  /** Resume from paused state. */
```

```
  resume(): Promise<RoomState>;

  /** Force-stop the room. */
  stop(): RoomState;

  /** Get the current FlowController instance. */
  getFlowController(): FlowController;

  /** Get the EventBus for this room. */
  getEventBus(): EventBus;
}
```

Design decisions: - Room is the unit of persistence. When a room is serialized (for restore or replay), it captures the full state snapshot. - `start()` is async because it triggers mode initialization: role assignment (may require LLM for random narrative), channel creation, and the first FlowController tick. - Room owns the agent list, channel list, and phase state. The FlowController reads and mutates phase state through the room.

3.4 Channel

Borrowed from AgentScope's `MsgHub` pattern. Channels control information flow -- who can see what.

```typescript
// packages/core/channel/types.ts

interface ChannelConfig {
  readonly id: string;
  readonly roomId: string;
  readonly name: string;
  readonly description?: string;
  /**
   * If true, messages sent to this channel are automatically delivered
   * to all subscribers. If false, messages are stored but subscribers
   * must explicitly poll (used for async channels).
   */
  readonly autoBroadcast: boolean;
  /** Parent channel ID for nested scoping. */
  readonly parentId: string | null;
  /** Default visibility for messages in this channel. */
  readonly defaultVisibility: MessageVisibility;
}

interface ChannelState {
  readonly config: ChannelConfig;
  readonly subscriberIds: readonly string[];
  readonly messageCount: number;
}

interface Channel {
  readonly state: ChannelState;

  /** Add an agent as a subscriber. */
  subscribe(agentId: string): ChannelState;

  /** Remove an agent from subscribers. */
  unsubscribe(agentId: string): ChannelState;

  /**
   * Publish a message to this channel.
   * If autoBroadcast is true, immediately calls observe() on all
   * eligible subscribers (respecting visibility masks).
   */
  publish(message: Message): Promise<void>;

  /** Get messages visible to a specific agent. */
```

```
  getMessagesForAgent(agentId: string, limit?: number): readonly Message

  /** Get all child channels (nested scoping). */
  getChildren(): readonly Channel[];
}
```

Design decisions: - **Nested scoping**: A Werewolf game has a `main` channel (everyone), a `werewolf-night` channel (werewolves only), and a `seer-check` channel (seer only). `werewolf-night` is a child of `main`. Messages in child channels are invisible to parent subscribers who are not subscribed to the child. - `autoBroadcast` **toggle**: When `true` (default), messages are pushed to subscribers immediately via `observe()`. When `false`, messages are stored and agents pull them (useful for asynchronous modes or delayed reveal mechanics like clue distribution in Script Kill). - **Dynamic subscription**: Agents can be subscribed/unsubscribed at runtime. When a Werewolf player is eliminated, they are unsubscribed from the `werewolf-night` channel but may be subscribed to a `spectator` channel. - **Visibility masks on messages**: Even within a channel, individual messages can have per-message visibility overrides. This handles edge cases like a seer's result being whispered only to the seer within the main channel.

Channel topology examples:

```
Roundtable:                    Werewolf:
  #main (all agents)             #main (all, day discussion)
                                   ├── #werewolf-night (werewolves only)
                                   ├── #seer-check (seer + system only)
                                   └── #spectator (eliminated + observers


Script Kill:                   TRPG:
  #main (all players)            #main (all players + GM)
  ├── #player-A (private)          ├── #gm-notes (GM only)
  ├── #player-B (private)          ├── #whisper-A (GM + player A)
  └── #investigation (phase-gated) └── #combat (active combatants)
```

3.5 FlowController

Controls turn order, phase transitions, and game state progression. Four implementations share one interface.

```
// packages/core/flow/types.ts

interface FlowTick {
  /** Which agent(s) should act next. Empty = waiting for external input
  readonly nextSpeakers: readonly string[];
  /** Instruction to pass to each speaker's reply() context. */
  readonly instruction?: string;
  /** Zod schema to constrain the agent's output (for decision turns). *
  readonly schema?: ZodSchema;
  /** Current phase name. */
  readonly phase: string;
  /** Is the flow complete? */
  readonly isComplete: boolean;
  /** Mode-specific tick data. */
  readonly metadata: Record<string, unknown>;
}

interface FlowController {
  /** Initialize with room state and mode config. */
  initialize(room: RoomState, config: Record<string, unknown>): void;

  /**
   * Advance the flow by one tick.
   * Called after each agent turn or external event.
   * Returns who goes next and what they should do.
   */
  tick(room: RoomState, lastMessage?: Message): FlowTick;

  /** Get the current phase. */
  getCurrentPhase(): string;

  /** Force a phase transition (used by mode lifecycle hooks). */
  forceTransition(phase: string): void;

  /** Check if the flow has reached a terminal state. */
  isComplete(): boolean;
}
```

FreeForm

Agents can speak whenever they want. No enforced turn order. A simple cooldown prevents spam.

```
// packages/core/flow/freeform.ts

interface FreeFormConfig {
  readonly cooldownMs: number;        // Min time between an agent's tur
  readonly maxTurnsPerAgent: number;  // Max turns per agent before forc
  readonly totalMaxTurns: number;     // Total turns before flow ends
}

// tick() returns ALL non-cooldown agents as nextSpeakers.
// The runtime picks one randomly or the first to respond.
```

Use case: Roundtable debate, brainstorming.

RoundRobin

Agents take turns in a fixed or randomized order. Supports multiple rounds.

```
// packages/core/flow/round-robin.ts

interface RoundRobinConfig {
  readonly order: 'fixed' | 'random' | 'randomPerRound';
  readonly rounds: number;           // Number of full rotations
  readonly skipEliminated: boolean;  // Skip agents with status 'elimina
}

// tick() returns the single next agent in the rotation.
// After all agents have gone, advances to next round.
```

Use case: Structured debate, initial introductions phase of a game.

StateMachine

Phase-driven flow with explicit transitions and per-phase rules. The workhorse for games.

```
// packages/core/flow/state-machine.ts

interface PhaseConfig {
  readonly name: string;
  readonly channels: readonly string[];     // Which channels are active
  readonly speakers: 'all' | 'role' | 'sequential' | 'custom';
  readonly speakerRoles?: readonly string[];  // If speakers = 'role'
  readonly speakerOrder?: readonly string[];  // If speakers = 'sequenti
  readonly schema?: ZodSchema;               // Decision schema for this
  readonly instruction?: string;              // Prompt instruction for
  readonly maxTurns?: number;                 // Auto-transition after N
  readonly timeoutMs?: number;                // Auto-transition after t
  readonly onEnter?: string;                  // Lifecycle hook name
  readonly onExit?: string;                   // Lifecycle hook name
}

interface TransitionRule {
  readonly from: string;
  readonly to: string;
  readonly condition: 'turnCount' | 'allSpoken' | 'vote' | 'custom';
  readonly params?: Record<string, unknown>;
}

interface StateMachineConfig {
  readonly phases: readonly PhaseConfig[];
  readonly transitions: readonly TransitionRule[];
  readonly initialPhase: string;
  readonly terminalPhases: readonly string[];
}

// tick() checks transition conditions after each message.
// When a transition fires, it calls onExit for current phase,
// switches state, calls onEnter for new phase.
```

Use case: Werewolf (night -> day discussion -> vote -> execution -> check win -> night), Script Kill.

Hierarchical

A top-level controller delegates to sub-controllers. Used for complex modes with nested flow patterns.

```
// packages/core/flow/hierarchical.ts

interface HierarchicalConfig {
  readonly stages: readonly {
    readonly name: string;
    readonly controller: 'freeform' | 'roundrobin' | 'statemachine';
    readonly config: FreeFormConfig | RoundRobinConfig | StateMachineCor
    readonly transitionCondition: 'complete' | 'turnCount' | 'custom';
    readonly transitionParams?: Record<string, unknown>;
  }[];
}

// tick() delegates to the current stage's sub-controller.
// When a stage completes, moves to the next stage.
```

Use case: TRPG (exploration phase uses FreeForm, combat uses RoundRobin, narrative events use StateMachine).

3.6 Memory

Two memory systems serve different purposes.

```
// packages/core/memory/types.ts

/**
 * SessionMemory: Short-term, per-session message history.
 * Lives in-memory during a room session, persisted to Postgres for rest
 * Uses LLM summarization to compress when context window fills up.
 */
interface SessionMemory {
  /** All raw messages this agent has observed. */
  readonly messages: readonly Message[];

  /** Compressed summaries of older messages. */
  readonly summaries: readonly MemorySummary[];

  /** Add a message to memory. Triggers compression if threshold exceede
  add(message: Message): SessionMemory;

  /**
   * Get messages formatted for LLM context.
   * Returns summaries + recent messages, fitting within tokenBudget.
   */
  getContext(tokenBudget: number): readonly ContextEntry[];

  /** Total token count of stored messages. */
  getTokenCount(): number;
}

interface MemorySummary {
  readonly id: string;
  readonly content: string;
  readonly messageRange: { readonly from: number; readonly to: number };
  readonly tokenCount: number;
  readonly createdAt: Date;
}

type ContextEntry =
  | { readonly type: 'summary'; readonly content: string }
  | { readonly type: 'message'; readonly message: Message };

/**
 * AgentLongTermMemory: Persistent across sessions.
 * Stores reflections and key experiences in pgvector.
```

```
 * Enables agents to "remember" past games and develop personality.
 */
interface AgentLongTermMemory {
  /** Record a memory (auto-embeds via LLM). */
  record(entry: MemoryEntry): Promise<void>;

  /** Retrieve relevant memories for a query (vector similarity search).
  retrieve(query: string, limit?: number): Promise<readonly MemoryEntry[

  /** Reflect on recent experiences to form higher-level insights. */
  reflect(recentEntries: readonly MemoryEntry[]): Promise<MemoryEntry>;

  /** Get the N most important memories (by importance score). */
  getTopMemories(n: number): Promise<readonly MemoryEntry[]>;
}

interface MemoryEntry {
  readonly id: string;
  readonly agentId: string;
  readonly content: string;
  readonly embedding: readonly number[];  // Vector embedding
  readonly importance: number;            // 0-10 importance score
  readonly type: 'observation' | 'reflection' | 'plan';
  readonly sessionId: string | null;      // Which room session this cam
  readonly createdAt: Date;
}
```

Design decisions: - SessionMemory is the agent's working memory for the current game. It holds raw messages and LLM-generated summaries. When the message count exceeds a threshold (configurable, default ~50 messages or ~4000 tokens), the oldest messages are summarized by the LLM and replaced with a compressed summary. - AgentLongTermMemory is optional. Not all modes need it. Script Kill and TRPG benefit from agents remembering past sessions. Roundtable debate does not. - The `reflect()` method is borrowed from the Stanford Generative Agents paper. After a session ends (or at periodic intervals), the agent generates higher-level reflections from recent observations. These reflections are stored as their own memory entries with type `'reflection'`. - Embeddings are generated via the LLM package and stored in pgvector for efficient similarity search.

3.7 EventBus

Platform events for UI subscription and mode lifecycle hooks.

```
// packages/core/events/types.ts

type PlatformEvent =
  | { type: 'agent:joined'; agentId: string; roomId: string }
  | { type: 'agent:left'; agentId: string; roomId: string }
  | { type: 'agent:statusChanged'; agentId: string; status: AgentState['
  | { type: 'message:sent'; message: Message }
  | { type: 'message:streamed'; messageId: string; chunk: string }
  | { type: 'phase:changed'; roomId: string; from: string; to: string }
  | { type: 'room:started'; roomId: string }
  | { type: 'room:paused'; roomId: string }
  | { type: 'room:completed'; roomId: string; result: Record<string, unk
  | { type: 'vote:cast'; agentId: string; vote: Record<string, unknown>
  | { type: 'vote:result'; roomId: string; result: Record<string, unknow
  | { type: 'channel:created'; channel: ChannelState }
  | { type: 'channel:subscriptionChanged'; channelId: string; agentId: s
  | { type: 'mode:custom'; eventName: string; data: Record<string, unkno

type EventHandler = (event: PlatformEvent) => void | Promise<void>;

interface EventBus {
  /** Subscribe to a specific event type. Returns unsubscribe function.
  on(type: PlatformEvent['type'], handler: EventHandler): () => void;

  /** Subscribe to all events. Returns unsubscribe function. */
  onAny(handler: EventHandler): () => void;

  /** Emit an event to all subscribers. */
  emit(event: PlatformEvent): void;

  /** Remove all handlers (cleanup on room destroy). */
  clear(): void;
}
```

Design decisions: - The EventBus is per-room. Each room has its own isolated event stream. - Events are the bridge between Platform Core and the frontend. The Socket.io server subscribes to the room's EventBus and forwards events to connected clients (respecting channel visibility). - `mode:custom` is the escape hatch. Modes can define their own event types without modifying the platform event taxonomy. - `message:streamed` supports token-by-token streaming for the UI. The LLM layer emits stream chunks as events.

3.8 StructuredOutput

Zod-based schema constraint for agent decisions. Borrowed from AgentScope's Pydantic pattern, adapted for TypeScript.

```
// packages/core/structured/types.ts

import { z, ZodSchema } from 'zod';

/**
 * Wraps a Zod schema with metadata for the LLM and UI.
 */
interface DecisionSchema<T extends ZodSchema = ZodSchema> {
  readonly name: string;
  readonly description: string;
  readonly schema: T;
}

// --- Common decision schemas ---

const VoteSchema = z.object({
  target: z.string().describe('The agent ID being voted for'),
  reason: z.string().describe('Brief justification for this vote'),
  confidence: z.number().min(0).max(1).describe('How confident the agent
});
type Vote = z.infer<typeof VoteSchema>;

const ActionSchema = z.object({
  action: z.string().describe('The action to take'),
  target: z.string().optional().describe('Target of the action (agent ID
  parameters: z.record(z.unknown()).optional().describe('Additional acti
});
type Action = z.infer<typeof ActionSchema>;

const SpeechWithDecisionSchema = z.object({
  speech: z.string().describe('What the agent says publicly'),
  decision: z.record(z.unknown()).optional().describe('Private structure
});
type SpeechWithDecision = z.infer<typeof SpeechWithDecisionSchema>;
```

Design decisions: - The Vercel AI SDK's `generateObject()` accepts a Zod schema directly. We pass the mode's decision schema through the FlowController tick, into the agent's reply context, and down to the LLM layer. - Each mode defines its own decision schemas. The platform provides common ones (Vote, Action) as starting points. - `SpeechWithDecisionSchema` solves a common pattern: the agent says something publicly but also makes a private decision (e.g., a werewolf says "I think player 3 is suspicious" but privately votes to kill player 2).

4. Mode System

4.1 Mode Interface

A mode is a plugin that provides everything needed to run a specific interaction pattern.

```
// packages/modes/types.ts

import { ZodSchema } from 'zod';

interface RoleTemplate {
  readonly id: string;
  readonly name: string;
  readonly description: string;
  readonly systemPromptTemplate: string;   // Supports {{variable}} inte
  readonly count: number | { min: number; max: number };
  readonly isRequired: boolean;
}

interface ChannelRule {
  readonly id: string;
  readonly name: string;
  readonly autoBroadcast: boolean;
  readonly parentId: string | null;
  /** Which roles have access. '*' = all. */
  readonly subscriberRoles: readonly string[] | '*';
  /** When is this channel active? '*' = always. */
  readonly activePhases: readonly string[] | '*';
}

interface UIExtension {
  /** Unique identifier for this UI extension. */
  readonly id: string;
  /** Where in the UI this extension renders. */
  readonly slot: 'sidebar' | 'header' | 'overlay' | 'chatFooter' | 'phas
  /** React component path (dynamic import). */
  readonly component: string;
  /** When this extension is visible. '*' = always. */
  readonly activePhases: readonly string[] | '*';
}

interface ModeLifecycleHooks {
  /** Called when the room starts. Assign roles, create channels, set ir
  onStart?: (room: RoomState) => Promise<RoomState>;
  /** Called when a phase changes. */
  onPhaseEnter?: (room: RoomState, phase: string) => Promise<RoomState>;
  onPhaseExit?: (room: RoomState, phase: string) => Promise<RoomState>;
  /** Called after each message. Can trigger side effects (clue reveal,
```

```
  onMessage?: (room: RoomState, message: Message) => Promise<RoomState>;
  /** Called when a vote completes. Process results (elimination, decisi
  onVoteComplete?: (room: RoomState, votes: readonly Vote[]) => Promise<
  /** Called when the mode ends. Compute results, award scores. */
  onComplete?: (room: RoomState) => Promise<{ result: Record<string, unk
}

interface Mode {
  readonly id: string;
  readonly name: string;
  readonly description: string;
  readonly minAgents: number;
  readonly maxAgents: number;

  /** Role templates available in this mode. */
  readonly roles: readonly RoleTemplate[];

  /** Flow controller configuration. */
  readonly flowConfig: {
    readonly type: 'freeform' | 'roundrobin' | 'statemachine' | 'hierarc
    readonly config: FreeFormConfig | RoundRobinConfig | StateMachineCon
  };

  /** Channel rules. */
  readonly channels: readonly ChannelRule[];

  /** Decision schemas used in this mode. */
  readonly schemas: Record<string, DecisionSchema>;

  /** UI extensions (optional). */
  readonly uiExtensions?: readonly UIExtension[];

  /** Lifecycle hooks. */
  readonly hooks: ModeLifecycleHooks;

  /** Validate mode-specific config (agent count, settings). */
  validateConfig(config: Record<string, unknown>): { valid: boolean; err
}
```

4.2 Mode Registration

Modes are discovered at build time via the `packages/modes/index.ts` registry:

```
// packages/modes/index.ts

import { roundtableMode } from './roundtable';
import { werewolfMode } from './werewolf';

const modeRegistry: Record<string, Mode> = {
  roundtable: roundtableMode,
  werewolf: werewolfMode,
  // script-kill, trpg added later
};

export function getMode(id: string): Mode {
  const mode = modeRegistry[id];
  if (!mode) throw new Error(`Unknown mode: ${id}`);
  return mode;
}

export function listModes(): readonly Mode[] {
  return Object.values(modeRegistry);
}
```

4.3 Example: Roundtable Mode

The simplest mode. Agents debate a topic in rounds, then vote on the best argument.

```
// packages/modes/roundtable/index.ts

import { z } from 'zod';
import type { Mode } from '../types';

const RoundtableVoteSchema = z.object({
  bestArguer: z.string().describe('Agent ID who made the strongest argum
  reason: z.string().describe('Why this agent had the best argument'),
});

export const roundtableMode: Mode = {
  id: 'roundtable',
  name: 'Roundtable Debate',
  description: 'Multiple AI agents debate a topic, then vote on the strc
  minAgents: 2,
  maxAgents: 8,

  roles: [
    {
      id: 'debater',
      name: 'Debater',
      description: 'Participates in the debate with a unique perspective
      systemPromptTemplate: `You are {{name}}, a debater in a roundtable
Your persona: {{persona}}
Topic: {{topic}}
Express your views clearly, challenge others constructively, and be oper
      count: { min: 2, max: 8 },
      isRequired: true,
    },
  ],

  flowConfig: {
    type: 'roundrobin',
    config: {
      order: 'randomPerRound',
      rounds: 3,
      skipEliminated: false,
    } satisfies RoundRobinConfig,
  },

  channels: [
    {
```

```
      id: 'main',
      name: 'Main Discussion',
      autoBroadcast: true,
      parentId: null,
      subscriberRoles: '*',
      activePhases: '*',
    },
  ],

  schemas: {
    vote: {
      name: 'RoundtableVote',
      description: 'Vote for the agent with the best argument',
      schema: RoundtableVoteSchema,
    },
  },

  hooks: {
    async onStart(room) {
      // All agents get 'debater' role, subscribe to main channel
      const updatedAgents = room.agents.map(agent => ({
        ...agent,
        role: 'debater',
        channelIds: ['main'],
      }));
      return { ...room, agents: updatedAgents, currentPhase: 'debate' };
    },

    async onVoteComplete(room, votes) {
      // Tally votes, declare winner
      const tally: Record<string, number> = {};
      for (const vote of votes) {
        const parsed = RoundtableVoteSchema.parse(vote);
        tally[parsed.bestArguer] = (tally[parsed.bestArguer] ?? 0) + 1;
      }
      const winner = Object.entries(tally).sort((a, b) => b[1] - a[1])[0
      return {
        ...room,
        currentPhase: 'complete',
        phaseData: { ...room.phaseData, winner: winner[0], tally },
      };
    },
```

```
    async onComplete(room) {
      return {
        result: { winner: room.phaseData.winner, tally: room.phaseData.t
        roomState: { ...room, status: 'completed', completedAt: new Date
      };
    },
  },

  validateConfig(config) {
    const errors: string[] = [];
    if (!config.topic || typeof config.topic !== 'string') {
      errors.push('A debate topic is required.');
    }
    return { valid: errors.length === 0, errors };
  },
};
```

4.4 Example: Werewolf Mode

Demonstrates StateMachine flow + Channel isolation + multiple roles + decision schemas.

```
// packages/modes/werewolf/index.ts

import { z } from 'zod';
import type { Mode, StateMachineConfig } from '../types';

// --- Decision Schemas ---

const WerewolfKillSchema = z.object({
  target: z.string().describe('Agent ID to kill tonight'),
  reason: z.string().describe('Why this target was chosen'),
});

const SeerCheckSchema = z.object({
  target: z.string().describe('Agent ID to investigate'),
});

const DayVoteSchema = z.object({
  target: z.string().describe('Agent ID to eliminate (or "skip" to not v
  reason: z.string().describe('Public justification for the vote'),
});

const LastWordsSchema = z.object({
  speech: z.string().describe('Final words before elimination'),
  reveal: z.boolean().describe('Whether to reveal your role'),
});

// --- Flow Configuration ---

const werewolfFlow: StateMachineConfig = {
  initialPhase: 'night',
  terminalPhases: ['werewolvesWin', 'villageWin'],
  phases: [
    {
      name: 'night',
      channels: ['werewolf-night', 'seer-check'],
      speakers: 'role',
      speakerRoles: ['werewolf'],
      schema: WerewolfKillSchema,
      instruction: 'Discuss with your fellow werewolves and choose a vil
      maxTurns: 6,
      onEnter: 'onNightStart',
    },
```

```
    {
      name: 'seerPhase',
      channels: ['seer-check'],
      speakers: 'role',
      speakerRoles: ['seer'],
      schema: SeerCheckSchema,
      instruction: 'Choose one player to investigate. You will learn whe
      maxTurns: 1,
    },
    {
      name: 'dayAnnouncement',
      channels: ['main'],
      speakers: 'all',
      instruction: 'The sun rises. Listen to the announcement of last ni
      maxTurns: 0, // System message only
      onEnter: 'onDayAnnouncement',
    },
    {
      name: 'dayDiscussion',
      channels: ['main'],
      speakers: 'sequential',
      instruction: 'Discuss who you think the werewolves are. Share your
      maxTurns: 12,
    },
    {
      name: 'dayVote',
      channels: ['main'],
      speakers: 'all',
      schema: DayVoteSchema,
      instruction: 'Vote for who to eliminate. You may also choose to sk
      onExit: 'onVoteTally',
    },
    {
      name: 'lastWords',
      channels: ['main'],
      speakers: 'custom', // Set dynamically to the eliminated player
      schema: LastWordsSchema,
      instruction: 'You have been voted out. Share your last words.',
      maxTurns: 1,
      onExit: 'onEliminationComplete',
    },
  ],
  transitions: [
```

```
    { from: 'night', to: 'seerPhase', condition: 'allSpoken' },
    { from: 'seerPhase', to: 'dayAnnouncement', condition: 'allSpoken' }
    { from: 'dayAnnouncement', to: 'dayDiscussion', condition: 'turnCour
    { from: 'dayDiscussion', to: 'dayVote', condition: 'turnCount', para
    { from: 'dayVote', to: 'lastWords', condition: 'allSpoken' },
    { from: 'lastWords', to: 'night', condition: 'custom', params: { hoc
  ],
};

// --- Mode Definition ---

export const werewolfMode: Mode = {
  id: 'werewolf',
  name: 'Werewolf (狼人杀)',
  description: 'Classic social deduction game. Werewolves hunt villagers
  minAgents: 6,
  maxAgents: 12,

  roles: [
    {
      id: 'werewolf',
      name: 'Werewolf',
      description: 'Kill one villager each night. Blend in during the da
      systemPromptTemplate: `You are {{name}}, a Werewolf in a game of W
Your persona: {{persona}}
You must eliminate villagers at night while appearing innocent during da
Your fellow werewolves are: {{teammates}}.
Strategy: coordinate kills at night, deflect suspicion during the day, a
      count: { min: 2, max: 4 },
      isRequired: true,
    },
    {
      id: 'villager',
      name: 'Villager',
      description: 'Find and vote out the werewolves before they elimina
      systemPromptTemplate: `You are {{name}}, a Villager in a game of W
Your persona: {{persona}}
Your goal: identify the werewolves through discussion and vote them out.
Pay attention to inconsistencies, suspicious behavior, and voting patter
      count: { min: 3, max: 7 },
      isRequired: true,
    },
    {
```

```
      id: 'seer',
      name: 'Seer',
      description: 'Each night, learn if one player is a werewolf. Use t
      systemPromptTemplate: `You are {{name}}, the Seer in a game of Wer
Your persona: {{persona}}
Each night you may investigate one player to learn if they are a werewol
Be careful about revealing your role -- werewolves will target you if th
      count: 1,
      isRequired: true,
    },
  ],

  flowConfig: {
    type: 'statemachine',
    config: werewolfFlow,
  },

  channels: [
    {
      id: 'main',
      name: 'Village Square',
      autoBroadcast: true,
      parentId: null,
      subscriberRoles: '*',
      activePhases: ['dayAnnouncement', 'dayDiscussion', 'dayVote', 'las
    },
    {
      id: 'werewolf-night',
      name: 'Werewolf Den',
      autoBroadcast: true,
      parentId: 'main',
      subscriberRoles: ['werewolf'],
      activePhases: ['night'],
    },
    {
      id: 'seer-check',
      name: 'Seer Vision',
      autoBroadcast: true,
      parentId: 'main',
      subscriberRoles: ['seer'],
      activePhases: ['seerPhase'],
    },
    {
```

```
      id: 'spectator',
      name: 'Spectator Gallery',
      autoBroadcast: true,
      parentId: null,
      subscriberRoles: [],  // Dynamically populated with eliminated pla
      activePhases: '*',
    },
  ],

  schemas: {
    kill: { name: 'WerewolfKill', description: 'Choose a target to elimi
    seerCheck: { name: 'SeerCheck', description: 'Choose a player to inv
    dayVote: { name: 'DayVote', description: 'Vote to eliminate a player
    lastWords: { name: 'LastWords', description: 'Final words before eli
  },

  uiExtensions: [
    {
      id: 'werewolf-phase-indicator',
      slot: 'phaseIndicator',
      component: './components/WerewolfPhaseIndicator',
      activePhases: '*',
    },
    {
      id: 'werewolf-vote-panel',
      slot: 'overlay',
      component: './components/WerewolfVotePanel',
      activePhases: ['dayVote'],
    },
    {
      id: 'werewolf-role-reveal',
      slot: 'sidebar',
      component: './components/WerewolfRoleSidebar',
      activePhases: '*',
    },
  ],

  hooks: {
    async onStart(room) {
      // Randomly assign roles, create channels, set up night phase
      const shuffled = [...room.agents].sort(() => Math.random() - 0.5);
      const roleAssignment = assignRoles(shuffled, room.config.modeConfi
      const updatedAgents = roleAssignment.map(({ agent, role, channels
```

```
        ...agent,
        role,
        channelIds: channels,
      }));
      return { ...room, agents: updatedAgents, currentPhase: 'night' };
    },

    async onVoteComplete(room, votes) {
      const tally = tallyVotes(votes, DayVoteSchema);
      const eliminated = getTopVoted(tally, room.agents);
      if (eliminated) {
        const updatedAgents = room.agents.map(a =>
          a.id === eliminated
            ? { ...a, status: 'eliminated' as const, channelIds: ['spect
            : a
        );
        return { ...room, agents: updatedAgents, phaseData: { ...room.ph
      }
      return room; // No elimination (tie or skip)
    },

    async onComplete(room) {
      const werewolves = room.agents.filter(a => a.role === 'werewolf');
      const alive = room.agents.filter(a => a.status !== 'eliminated');
      const werewolvesAlive = alive.filter(a => a.role === 'werewolf');
      const villagersAlive = alive.filter(a => a.role !== 'werewolf');

      const winner = werewolvesAlive.length >= villagersAlive.length ? '
      return {
        result: { winner, survivors: alive.map(a => a.id) },
        roomState: { ...room, status: 'completed', completedAt: new Date
      };
    },
  },

  validateConfig(config) {
    const errors: string[] = [];
    const agentCount = (config.agentCount as number) ?? 0;
    if (agentCount < 6) errors.push('Werewolf requires at least 6 agents
    if (agentCount > 12) errors.push('Werewolf supports at most 12 agent
    return { valid: errors.length === 0, errors };
```

```
  },
};
```

5. Frontend Architecture

5.1 Page Structure

```
apps/web/app/
├── layout.tsx                    # Root layout (providers, theme)
├── page.tsx                      # Landing / lobby
├── (lobby)/
│   ├── rooms/
│   │   └── page.tsx              # Room list
│   └── create/
│       └── page.tsx              # Create room wizard
├── room/
│   └── [id]/
│       ├── layout.tsx            # Room layout (Socket.io connection)
│       ├── page.tsx              # Room view (redirects based on room s
│       ├── waiting/
│       │   └── page.tsx          # Waiting room (add agents, configure)
│       └── play/
│           └── page.tsx          # Active room (game/session view)
└── api/
    ├── rooms/
    │   └── route.ts              # Room CRUD
    ├── agents/
    │   └── route.ts              # Agent management
    └── socket/
        └── route.ts              # Socket.io server initialization
```

5.2 Real-time Communication

```
                  +-----------+
                  |  Browser  |
                  +-----+-----+
                        |
                  Socket.io Client
                        |
         +-------------+-------------+
         |                           |
   Client -> Server            Server -> Client
   ===============            ================
   humanMessage                message
   humanVote                   messageStream
   roomConfig                  phaseChange
   joinRoom                    agentStatusChange
   leaveRoom                   voteResult
   pauseRoom                   agentJoined
   resumeRoom                  agentLeft
                               roomStateSync
                               error
                               modeCustomEvent
```

The Socket.io server subscribes to the room's EventBus and translates PlatformEvents into Socket.io events. Channel-aware broadcasting ensures clients only receive events they are authorized to see.

```
// Simplified Socket.io server wiring

io.on('connection', (socket) => {
  socket.on('joinRoom', async ({ roomId, userId }) => {
    socket.join(roomId);

    const room = await getRoom(roomId);
    const eventBus = room.getEventBus();

    // Subscribe to room events, filter by user's visibility
    eventBus.onAny((event) => {
      if (event.type === 'message:sent') {
        // Only send if user is allowed to see this message
        if (isVisibleToUser(event.message, userId, room)) {
          socket.emit('message', event.message);
        }
      } else {
        socket.emit(event.type, event);
      }
    });

    // Send initial room state
    socket.emit('roomStateSync', sanitizeRoomState(room.state, userId));
  });

  socket.on('humanMessage', async ({ roomId, content }) => {
    // Handle human player input
    const room = await getRoom(roomId);
    const humanAgent = getHumanAgent(room, socket.userId);
    if (humanAgent) {
      await humanAgent.submitHumanInput(content);
    }
  });
});
```

5.3 Key Components

```
+-----------------------------------------------------------------------
|  RoomView
|  +-------------------+  +-----------------------------------------+ |
|  |  Sidebar          |  |  MainContent                              | |
|  |  +-------------+  |  |  +-----------------------------------+  | |
|  |  | AgentList   |  |  |  | PhaseIndicator                    |  | |
|  |  | - Avatar    |  |  |  | [Night] -> [Day] -> [Vote]        |  | |
|  |  | - Name      |  |  |  +-----------------------------------+  | |
|  |  | - Status    |  |  |                                         | |
|  |  | - Role(*)   |  |  |  +-----------------------------------+  | |
|  |  +-------------+  |  |  | ChatFeed                           |  | |
|  |                    |  |  | +-------------------------------+ |  |
|  |  +-------------+  |  |  | | ChatBubble (agent message)     | |  | |
|  |  | ModePanel   |  |  |  | |  [Avatar] Name           time  | |  | |
|  |  | (UIExtension|  |  |  | |  Message content...            | |  | |
|  |  |  slot:      |  |  |  | +-------------------------------+ |  | |
|  |  |  sidebar)   |  |  |  | | ChatBubble (system message)    | |  | |
|  |  +-------------+  |  |  | |  Phase changed to Day          | |  | |
|  |                    |  |  | +-------------------------------+ |  |
|  |  +-------------+  |  |  | | ChatBubble (agent thinking...) | |  | |
|  |  | RoomInfo    |  |  |  | |  [streaming indicator]         | |  | |
|  |  | - Mode      |  |  |  | +-------------------------------+ |  | |
|  |  | - Phase     |  |  |  +-----------------------------------+  | |
|  |  | - Timer     |  |  |                                         | |
|  |  +-------------+  |  |  +-----------------------------------+  | |
|  +-------------------+  |  | InputArea (human players only)     |  | |
|                          |  | [Type your message...]       [Send]|  |
|                          |  +-----------------------------------+  |
|                          |                                         |
|                          |  +-----------------------------------+  |
|                          |  | VotePanel (UIExtension overlay)    |  |
|                          |  | [Agent A] [Agent B] [Agent C] [Skip]| |
|                          |  +-----------------------------------+  |
|                          +-----------------------------------------+
+-----------------------------------------------------------------------
```

5.4 State Management

Client-side state is managed with a combination of React Context and lightweight stores:

```
// Simplified state architecture

// 1. RoomContext -- provided by room/[id]/layout.tsx
interface RoomContextValue {
  readonly room: RoomState;
  readonly socket: Socket;
  readonly currentUserId: string;
}

// 2. useRoomState hook -- syncs with server via Socket.io
function useRoomState(roomId: string) {
  const [room, setRoom] = useState<RoomState | null>(null);
  const socket = useSocket();

  useEffect(() => {
    socket.emit('joinRoom', { roomId });
    socket.on('roomStateSync', setRoom);
    socket.on('phaseChange', ({ from, to }) => {
      setRoom(prev => prev ? { ...prev, currentPhase: to } : null);
    });
    socket.on('agentStatusChange', ({ agentId, status }) => {
      setRoom(prev => prev ? updateAgentStatus(prev, agentId, status) :
    });
    // ... other event handlers
    return () => { socket.emit('leaveRoom', { roomId }); };
  }, [roomId]);

  return room;
}

// 3. useMessages hook -- message feed with channel filtering
function useMessages(roomId: string, channelId?: string) {
  const [messages, setMessages] = useState<Message[]>([]);
  const socket = useSocket();

  useEffect(() => {
    socket.on('message', (msg: Message) => {
      if (!channelId || msg.channelId === channelId) {
        setMessages(prev => [...prev, msg]);
      }
    });
  }, [roomId, channelId]);
```

```
  return messages;
}
```

5.5 Mode UI Extensions

Modes can inject custom UI components into predefined slots. The frontend dynamically loads these components:

```
// apps/web/components/ModeExtensionSlot.tsx

interface ModeExtensionSlotProps {
  readonly slot: UIExtension['slot'];
  readonly mode: Mode;
  readonly phase: string;
  readonly room: RoomState;
}

function ModeExtensionSlot({ slot, mode, phase, room }: ModeExtensionSlo
  const extensions = (mode.uiExtensions ?? []).filter(
    ext => ext.slot === slot && (ext.activePhases === '*' || ext.activeP
  );

  return (
    <>
      {extensions.map(ext => {
        const Component = dynamic(() => import(`@agora/modes/${mode.id}/
        return <Component key={ext.id} room={room} phase={phase} />;
      })}
    </>
  );
}
```

6. LLM Integration

6.1 Vercel AI SDK Usage Pattern

```
// packages/llm/provider.ts

import { createOpenAI } from '@ai-sdk/openai';
import { createAnthropic } from '@ai-sdk/anthropic';
import { createGoogleGenerativeAI } from '@ai-sdk/google';

type ModelProvider = 'openai' | 'anthropic' | 'google';

interface ModelConfig {
  readonly provider: ModelProvider;
  readonly modelId: string;
  readonly apiKey: string;
  readonly baseURL?: string;
}

function createProvider(config: ModelConfig) {
  switch (config.provider) {
    case 'openai':
      return createOpenAI({ apiKey: config.apiKey, baseURL: config.baseU
    case 'anthropic':
      return createAnthropic({ apiKey: config.apiKey });
    case 'google':
      return createGoogleGenerativeAI({ apiKey: config.apiKey });
  }
}

// Model registry -- each agent can use a different model
const modelRegistry = new Map<string, ModelConfig>();

function registerModel(agentId: string, config: ModelConfig): void {
  modelRegistry.set(agentId, config);
}

function getModelForAgent(agentId: string) {
  const config = modelRegistry.get(agentId);
  if (!config) throw new Error(`No model configured for agent: ${agentIc
  const provider = createProvider(config);
  return provider(config.modelId);
}
```

6.2 Text and Structured Generation

```
// packages/llm/generate.ts

import { generateText, generateObject, streamText } from 'ai';
import { ZodSchema } from 'zod';

interface GenerateOptions {
  readonly agentId: string;
  readonly systemPrompt: string;
  readonly messages: readonly ContextEntry[];
  readonly temperature?: number;
  readonly maxTokens?: number;
}

/**
 * Generate a free-text response.
 */
async function generateResponse(options: GenerateOptions): Promise<{
  readonly text: string;
  readonly tokenCount: { prompt: number; completion: number };
}> {
  const model = getModelForAgent(options.agentId);
  const result = await generateText({
    model,
    system: options.systemPrompt,
    messages: formatMessages(options.messages),
    temperature: options.temperature ?? 0.7,
    maxTokens: options.maxTokens ?? 1024,
  });
  return {
    text: result.text,
    tokenCount: {
      prompt: result.usage.promptTokens,
      completion: result.usage.completionTokens,
    },
  };
}

/**
 * Generate a structured decision constrained by a Zod schema.
 * Uses Vercel AI SDK's generateObject() which maps to tool calling
```

```
 * or JSON mode depending on the provider.
 */
async function generateDecision<T>(
  options: GenerateOptions,
  schema: ZodSchema<T>,
): Promise<{
  readonly decision: T;
  readonly tokenCount: { prompt: number; completion: number };
}> {
  const model = getModelForAgent(options.agentId);
  const result = await generateObject({
    model,
    schema,
    system: options.systemPrompt,
    messages: formatMessages(options.messages),
    temperature: options.temperature ?? 0.3,  // Lower temp for decision
    maxTokens: options.maxTokens ?? 512,
  });
  return {
    decision: result.object,
    tokenCount: {
      prompt: result.usage.promptTokens,
      completion: result.usage.completionTokens,
    },
  };
}
```

6.3 Streaming Strategy

Streaming is used for the UI to show agents "thinking" in real-time:

```
// packages/llm/stream.ts

import { streamText } from 'ai';

async function streamResponse(
  options: GenerateOptions,
  onChunk: (chunk: string) => void,
): Promise<{ text: string; tokenCount: { prompt: number; completion: num
  const model = getModelForAgent(options.agentId);
  const result = streamText({
    model,
    system: options.systemPrompt,
    messages: formatMessages(options.messages),
    temperature: options.temperature ?? 0.7,
    maxTokens: options.maxTokens ?? 1024,
  });

  let fullText = '';
  for await (const chunk of result.textStream) {
    fullText += chunk;
    onChunk(chunk);   // EventBus emits 'message:streamed'
  }

  const usage = await result.usage;
  return {
    text: fullText,
    tokenCount: { prompt: usage.promptTokens, completion: usage.completi
  };
}
```

When streaming is used vs not: - **Streaming**: Agent speech during discussion phases (UI shows typing indicator). - **Non-streaming (generateObject)**: Structured decisions (votes, actions). These are atomic -- no partial output is useful.

6.4 Token Management and Cost Awareness

```
// packages/llm/cost.ts

interface TokenUsage {
  readonly agentId: string;
  readonly roomId: string;
  readonly promptTokens: number;
  readonly completionTokens: number;
  readonly estimatedCostUsd: number;
}

// Cost per 1M tokens (approximate, varies by model)
const COST_TABLE: Record<string, { input: number; output: number }> = {
  'claude-sonnet-4-20250514':       { input: 3.0,  output: 15.0 },
  'claude-haiku-3.5':               { input: 0.8,  output: 4.0 },
  'gpt-4o':                         { input: 2.5,  output: 10.0 },
  'gpt-4o-mini':                    { input: 0.15, output: 0.6 },
  'gemini-2.0-flash':               { input: 0.1,  output: 0.4 },
};

function estimateCost(modelId: string, promptTokens: number, completionT
  const costs = COST_TABLE[modelId] ?? { input: 5.0, output: 15.0 };
  return (promptTokens * costs.input + completionTokens * costs.output)
}

// Room-level cost tracking
interface RoomCostTracker {
  recordUsage(usage: TokenUsage): void;
  getTotalCost(): number;
  getCostByAgent(): Record<string, number>;
  isOverBudget(budgetUsd: number): boolean;
}
```

7. Data Model

7.1 Postgres Schema

```
-- Rooms
CREATE TABLE rooms (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  mode_id     TEXT NOT NULL,
  mode_config JSONB NOT NULL DEFAULT '{}',
  status      TEXT NOT NULL DEFAULT 'waiting'
                CHECK (status IN ('waiting', 'running', 'paused', 'compl
  created_by  TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  started_at  TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  result      JSONB
);

-- Agents (per-room instance of an agent)
CREATE TABLE room_agents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id     UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  name        TEXT NOT NULL,
  persona     TEXT NOT NULL,
  model_id    TEXT NOT NULL,
  role        TEXT,                       -- Mode-assigned role
  status      TEXT NOT NULL DEFAULT 'idle'
                CHECK (status IN ('idle', 'thinking', 'speaking', 'obser
  is_human    BOOLEAN NOT NULL DEFAULT false,
  config      JSONB NOT NULL DEFAULT '{}',
  metadata    JSONB NOT NULL DEFAULT '{}',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),

  UNIQUE (room_id, name)
);

-- Channels
CREATE TABLE channels (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id         UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  name            TEXT NOT NULL,
  auto_broadcast  BOOLEAN NOT NULL DEFAULT true,
  parent_id       UUID REFERENCES channels(id),
```

```
  config          JSONB NOT NULL DEFAULT '{}',

  UNIQUE (room_id, name)
);

-- Channel subscriptions
CREATE TABLE channel_subscriptions (
  channel_id  UUID NOT NULL REFERENCES channels(id) ON DELETE CASCADE,
  agent_id    UUID NOT NULL REFERENCES room_agents(id) ON DELETE CASCADE
  subscribed_at TIMESTAMPTZ NOT NULL DEFAULT now(),

  PRIMARY KEY (channel_id, agent_id)
);

-- Messages
CREATE TABLE messages (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id     UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  channel_id  UUID NOT NULL REFERENCES channels(id) ON DELETE CASCADE,
  sender_id   UUID REFERENCES room_agents(id),   -- NULL for system mess
  sender_name TEXT NOT NULL,
  role        TEXT NOT NULL CHECK (role IN ('agent', 'system', 'human',
  content     TEXT NOT NULL,
  visibility  JSONB NOT NULL DEFAULT '{"broadcast": true}',
  metadata    JSONB NOT NULL DEFAULT '{}',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_messages_room_channel ON messages(room_id, channel_id,
CREATE INDEX idx_messages_room_sender ON messages(room_id, sender_id, cr

-- Session memory summaries (LLM-compressed)
CREATE TABLE session_summaries (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id       UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  agent_id      UUID NOT NULL REFERENCES room_agents(id) ON DELETE CASCA
  content       TEXT NOT NULL,
  message_range JSONB NOT NULL,        -- { from: number, to: number }
  token_count   INTEGER NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Agent long-term memories (with pgvector)
```

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE agent_memories (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id    TEXT NOT NULL,            -- Persistent agent identity (no
  content     TEXT NOT NULL,
  embedding   VECTOR(1536) NOT NULL,   -- OpenAI ada-002 or equivalent
  importance  REAL NOT NULL DEFAULT 5.0,
  type        TEXT NOT NULL CHECK (type IN ('observation', 'reflection',
  session_id  UUID REFERENCES rooms(id),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_agent_memories_agent ON agent_memories(agent_id, create
CREATE INDEX idx_agent_memories_embedding ON agent_memories
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);

-- Room state snapshots (for restore and replay)
CREATE TABLE room_snapshots (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id     UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  state       JSONB NOT NULL,           -- Full RoomState serialization
  phase       TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_room_snapshots ON room_snapshots(room_id, created_at DE
```

7.2 What is Persisted vs In-Memory

Data	Storage	Reason
Room config, status	Postgres	Durable, survives restarts
Agent config, role, status	Postgres	Durable
Messages	Postgres	Permanent record, replay

Data	Storage	Reason
Channel config, subscriptions	Postgres	Restore after restart
Session memory summaries	Postgres	Restore agent context
Long-term memories + embeddings	Postgres (pgvector)	Cross-session persistence
Room snapshots	Postgres	Session restore, replay
FlowController state	In-memory (snapshot to Postgres)	High-frequency reads, snapshot periodically
EventBus subscriptions	In-memory	Ephemeral, reconstructed on connect
Active Socket.io connections	In-memory	Ephemeral by nature
LLM streaming buffers	In-memory	Ephemeral, per-request

7.3 Session Restore Strategy

When a server restarts or a client reconnects:

1. Load the latest `room_snapshots` record for the room.
2. Reconstruct `RoomState` from the snapshot JSON.
3. Load any messages created after the snapshot timestamp from the `messages` table.
4. Replay those messages through the FlowController to rebuild current state.
5. Recreate Channel subscriptions from `channel_subscriptions`.
6. Rebuild each agent's SessionMemory from `session_summaries` + recent messages.

7. Resume the FlowController from the restored phase.

This "snapshot + replay" approach balances durability with performance. Snapshots are taken at phase transitions (natural checkpoints), so replay only covers the current phase's messages.

8. Real-time Communication

8.1 Socket.io Event Taxonomy

Server to Client Events

Event	Payload	When
`message`	`Message`	Agent or system sends a message
`messageStream`	`{ messageId, chunk }`	Streaming token from an agent
`phaseChange`	`{ from, to, phaseData }`	FlowController transitions phase
`agentStatusChange`	`{ agentId, status }`	Agent starts thinking, speaking, etc.
`agentJoined`	`{ agent: AgentState }`	Agent added to room
`agentLeft`	`{ agentId }`	Agent removed from room
`voteResult`	`{ tally, eliminated? }`	Vote phase completed
`roomStateSync`	`RoomState` (sanitized)	Full state on join or reconnect

Event	Payload	When
`roomStatusChange`	`{ status }`	Room started, paused, completed
`error`	`{ code, message }`	Error notification
`modeCustomEvent`	`{ eventName, data }`	Mode-specific event

Client to Server Events

Event	Payload	When
`joinRoom`	`{ roomId }`	Client opens room page
`leaveRoom`	`{ roomId }`	Client navigates away
`humanMessage`	`{ roomId, content }`	Human player sends a message
`humanVote`	`{ roomId, vote }`	Human player casts a vote
`humanAction`	`{ roomId, action }`	Human player takes a structured action
`roomConfig`	`{ roomId, config }`	Host updates room config
`startRoom`	`{ roomId }`	Host starts the session
`pauseRoom`	`{ roomId }`	Host pauses
`resumeRoom`	`{ roomId }`	Host resumes

8.2 Channel-Aware Broadcasting

The Socket.io server does not blindly broadcast all events. It filters based on channel visibility:

```
// Pseudocode for channel-aware event routing

function shouldClientReceiveMessage(
  message: Message,
  userId: string,
  room: RoomState,
): boolean {
  // System messages are always visible
  if (message.role === 'system') return true;

  // Check message-level visibility
  const vis = message.visibility;
  if (vis.denyList?.includes(userId)) return false;
  if (vis.allowList && !vis.allowList.includes(userId)) return false;

  // Check channel subscription
  const channel = room.channels.find(c => c.config.id === message.channe
  if (!channel) return false;

  // If user is a spectator (non-player), show all channels
  const userAgent = room.agents.find(a => a.config.isHuman && a.config.i
  if (!userAgent) return true; // Spectator sees everything

  // Check if user's agent is subscribed to this channel
  return channel.subscriberIds.includes(userAgent.id);
}
```

8.3 Reconnection Handling

```
Client reconnects
       |
       v
Send 'joinRoom' with lastMessageId
       |
       v
Server loads room state
       |
       v
Server sends 'roomStateSync' (full current state)
       |
       v
Server sends missed messages since lastMessageId
       |
       v
Client merges into local state
```

9. Memory System Deep Dive

9.1 SessionMemory: Compression Strategy

Session memory prevents context window overflow by compressing old messages into summaries.

```
Messages arrive via observe()
         |
         v
   messages[] grows
         |
   Token count > threshold?  (default: 4000 tokens)
         |
    NO   |   YES
    |    |    |
    v    |    v
  (wait) | Take oldest N messages (N = half the buffer)
         |    |
         v    v
         LLM summarizes them into 1-2 paragraphs
              |
              v
         Create MemorySummary {
           content: "summary text",
           messageRange: { from: 0, to: N },
           tokenCount: ~200
         }
              |
              v
         Replace N messages with summary in context
              |
              v
         getContext(tokenBudget) returns:
           [summary1, summary2, ..., recentMsg1, recentMsg2, ...]
```

Compression prompt template:

```
Summarize the following conversation from the perspective of {{agentName
Focus on: key arguments, decisions made, information revealed, and your
Be concise but preserve strategically important details.

Messages:
{{messages}}
```

When compression triggers: - After each `observe()` call, if `getTokenCount() > threshold`. - The threshold is configurable per mode. Games with many short turns (Werewolf) use a lower threshold. Debates with long arguments use a higher one.

9.2 AgentLongTermMemory: Record-Embed-Store-Retrieve Cycle

```
Session ends (or periodic trigger)
         |
         v
Extract key observations from session:
  - "Player A accused me of being a werewolf"
  - "My strategy of lying low worked until round 3"
  - "Seer revealed player B as werewolf on night 2"
         |
         v
For each observation:
  1. Score importance (LLM rates 1-10)
  2. Generate embedding (LLM embedding API)
  3. Store in agent_memories table
         |
         v
On next session start:
  1. Query: "What do I remember about playing werewolf?"
  2. Vector similarity search in pgvector
  3. Return top-k most relevant memories
  4. Inject into system prompt as "past experiences"
```

9.3 Reflect Mechanism

Borrowed from the Stanford Generative Agents paper. Reflections are higher-level insights synthesized from raw observations.

```
After every K observations (default K=10) or at session end:
         |
         v
Gather recent observations
         |
         v
LLM prompt:
  "Given these recent experiences, what are 2-3 higher-level
   insights or patterns you've noticed?"
         |
         v
Each reflection becomes a new MemoryEntry with:
  type: 'reflection'
  importance: (LLM-scored, typically higher than observations)
         |
         v
Stored in agent_memories with embedding
```

Example reflections: - Observation: "Player A voted for me twice. Player B defended me." - Reflection: "When I'm under suspicion, having an ally who defends me is crucial for survival. I should build alliances early."

9.4 Memory Activation Per Phase

Phase Type	SessionMemory	LongTermMemory
Discussion	Active (recent messages)	Retrieved if agent has past sessions
Night action	Active (werewolf chat)	Not typically used
Voting	Active (today's discussion)	May retrieve past voting patterns
Session start	Empty, initialized	Retrieved: "What do I know about this mode?"
Session end	Archived	Record observations + reflect

Phase Type	SessionMemory	LongTermMemory
Between sessions	Persisted as summaries	Persisted in pgvector

10. Security & Performance Considerations

10.1 API Key Management

Two models supported, selectable per deployment:

User's own keys (default, open-source mode): - User provides API keys via the room configuration UI. - Keys are stored in the browser's local storage or encrypted in Postgres per-user. - Keys are never logged or sent to third parties. - Each agent in a room can use a different provider/key.

Platform keys (hosted mode, future): - Platform provides API keys, charges users per-token. - Keys stored server-side, never exposed to clients. - Usage tracked per-room, per-agent.

```
// API key resolution order
function resolveApiKey(agentConfig: AgentConfig, roomConfig: RoomConfig)
  // 1. Agent-level key (specific model)
  if (agentConfig.apiKey) return agentConfig.apiKey;
  // 2. Room-level key (shared across agents)
  if (roomConfig.apiKeys?.[agentConfig.provider]) return roomConfig.apiK
  // 3. Platform-level key (env var)
  const envKey = process.env[`${agentConfig.provider.toUpperCase()}_API_
  if (envKey) return envKey;
  throw new Error(`No API key available for provider: ${agentConfig.prov
}
```

10.2 Rate Limiting

```
interface RateLimitConfig {
  /** Max LLM calls per agent per minute. */
  readonly agentCallsPerMinute: number;
  /** Max total LLM calls per room per minute. */
  readonly roomCallsPerMinute: number;
  /** Max tokens per agent per session. */
  readonly agentTokenBudget: number;
  /** Max tokens per room per session. */
  readonly roomTokenBudget: number;
}

const DEFAULT_RATE_LIMITS: RateLimitConfig = {
  agentCallsPerMinute: 10,
  roomCallsPerMinute: 60,
  agentTokenBudget: 100_000,
  roomTokenBudget: 500_000,
};
```

Rate limiting is enforced at the LLM layer. When a limit is hit: - Agent's turn is skipped with a system message ("Agent X is rate-limited, skipping turn"). - If room budget is exhausted, room is paused with an error event.

10.3 Concurrent Agent Execution

When multiple agents need to act simultaneously (e.g., all werewolves deciding on a target, or multiple agents voting):

```
FlowController.tick() returns nextSpeakers = [A, B, C]
         |
         v
Runtime creates Promise.allSettled([
  agentA.reply(context, schema),
  agentB.reply(context, schema),
  agentC.reply(context, schema),
])
         |
         v
Results collected, published in order, tick() called again
```

Concurrency rules: - Agents in the same phase can run in parallel (their context is frozen at tick time). - Each agent's `observe()` for another agent's message in the same batch happens after all replies complete. - LLM calls are I/O-bound, so parallelism is efficient even on a single server. - `Promise.allSettled` ensures one agent's LLM error does not block others.

10.4 Cost Estimation and Limits

Before a room starts, the system provides a cost estimate:

```
function estimateRoomCost(config: RoomConfig): {
  readonly estimatedMinUsd: number;
  readonly estimatedMaxUsd: number;
  readonly breakdown: Record<string, number>;
} {
  const mode = getMode(config.modeId);
  const agentCount = config.maxAgents;

  // Estimate based on mode's typical turn count and model costs
  const avgTurnsPerAgent = getAverageTurns(mode);
  const avgTokensPerTurn = 800;  // ~200 prompt + ~600 completion
  const totalTokens = agentCount * avgTurnsPerAgent * avgTokensPerTurn;

  // Calculate per-model costs
  // ... (uses COST_TABLE from section 6.4)

  return { estimatedMinUsd, estimatedMaxUsd, breakdown };
}
```

The UI shows this estimate before the user clicks "Start". Users can set a hard budget limit that triggers auto-pause.

10.5 Input Validation

All external inputs are validated with Zod at the API boundary:

```
// apps/web/api/rooms/route.ts

import { z } from 'zod';

const CreateRoomSchema = z.object({
  name: z.string().min(1).max(100),
  modeId: z.string(),
  modeConfig: z.record(z.unknown()),
  maxAgents: z.number().int().min(2).max(20),
  allowHumans: z.boolean().default(false),
});

export async function POST(req: Request) {
  const body = await req.json();
  const validated = CreateRoomSchema.parse(body);
  // ... create room
}
```

10.6 Information Isolation Enforcement

Channel visibility is enforced at multiple layers:

1. **Channel.publish()**: Only calls `observe()` on subscribers who pass the visibility check.
2. **Socket.io server**: Filters events before sending to clients (see section 8.2).
3. **API routes**: `GET /api/rooms/:id/messages` filters by the requesting user's channel access.
4. **Frontend**: Even if a message somehow reaches the client, the UI only renders messages for subscribed channels.

Defense in depth -- a bug in one layer does not leak information.

Appendix: Quick Reference

Architecture Layer Mapping

Concept	Layer	Package
Agent, Room, Channel	Platform Core	`packages/core`
FlowController	Platform Core	`packages/core`
Memory, EventBus	Platform Core	`packages/core`
Mode, RoleTemplate, ChannelRule	Mode Layer	`packages/modes`
LLM generation, streaming	Infrastructure	`packages/llm`
Socket.io, Postgres	Infrastructure	`apps/web`
Next.js pages, components	Infrastructure	`apps/web`
Shared types, Zod schemas	Cross-cutting	`packages/shared`

Key Patterns

Pattern	Origin	Agora Adaptation
`reply()` + `observe()`	AgentScope `AgentBase`	Same two-method agent contract
MsgHub channels	AgentScope `MsgHub`	Extended with nested scopes, visibility masks
Structured output	AgentScope Pydantic constraints	Zod schemas via Vercel AI SDK `generateObject()`
Generative memory	Stanford Generative Agents	pgvector for persistence, reflect mechanism
Mode plugins	Game modding patterns	TypeScript interface with roles, flow, channels, schemas, UI, hooks

Data Flow Summary

```
User -> Next.js API -> Room.start()
  -> Mode.hooks.onStart() -> assign roles, create channels
  -> FlowController.tick() -> pick next speaker(s)
  -> Agent.reply() -> LLM.generate() -> Message
  -> Channel.publish() -> eligible agents.observe()
  -> EventBus.emit() -> Socket.io -> Browser UI
  -> FlowController.tick() -> check transitions
  -> (loop until isComplete)
  -> Mode.hooks.onComplete() -> results
  -> Room status = 'completed'
```